

# Chapter 8: Deep Quantum Learning

---

Gi-Chan, Bae

June 3, 2022

Research Institute of Mathematical Science, Seoul National University  
gcbae02@snu.ac.kr

# Table of Contents

## Cosine function fitting

- Review of the neural network

- Construction of data and function

- Cosine function fitting by QNN

## Distinguish handwritten 1 and 7 by QNN

- Setting of the problem

- Training by QNN

- Comparison with CNN

# Table of Contents

## Cosine function fitting

- Review of the neural network

- Construction of data and function

- Cosine function fitting by QNN

## Distinguish handwritten 1 and 7 by QNN

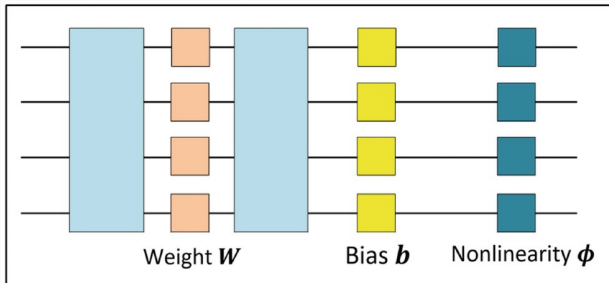
- Setting of the problem

- Training by QNN

- Comparison with CNN

# Classic Neural Network

- Classic Neural Network

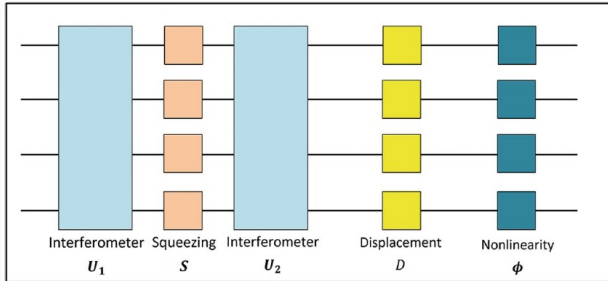


Classic Neural Network

$$z = f(x) \rightarrow \phi(\mathbf{W} \cdot \mathbf{x} + \mathbf{b})$$

# Quantum Neural Network

- Quantum Neural Network



Quantum Neural Network

Input:  $x$ -axis value

Output:  $\cos(x) + \text{noise}$

# Table of Contents

## Cosine function fitting

Review of the neural network

**Construction of data and function**

Cosine function fitting by QNN

## Distinguish handwritten 1 and 7 by QNN

Setting of the problem

Training by QNN

Comparison with CNN

# Install and Import

```
In [2]: pip install pennylane --upgrade
```

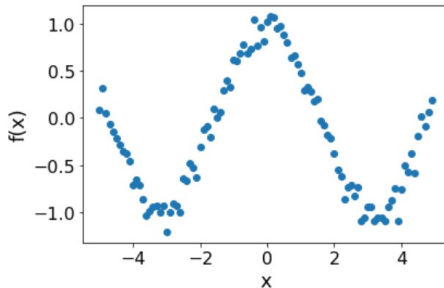
```
Requirement already satisfied: pennylane in c:\users\gcbae\anaconda3\lib\site-packages  
Requirement already satisfied: toml in c:\users\gcbae\anaconda3\lib\site-packages  
Requirement already satisfied: pennylane-lightning>=0.23 in c:\users\gcbae\anaconda3\lib\site-packages  
Requirement already satisfied: appdirs in c:\users\gcbae\anaconda3\lib\site-packages
```

```
In [2]: import numpy as np  
import pennylane as qml  
from pennylane import numpy as np  
from pennylane.optimize import AdamOptimizer  
import matplotlib.pyplot as plt  
import pylab
```

# Construction of the noise data

```
In [11]: Xlim = 5
noise = np.random.normal(0,0.1,100) # generate noise to add to the function
# define functions
X = np.arange(-Xlim, Xlim, 0.1)
Y = np.cos(X)+noise

plt.figure()
plt.scatter(X, Y)
plt.xlabel("x", fontsize=18)
plt.ylabel("f(x)", fontsize=18)
plt.tick_params(axis="both", which="major", labelsize=16)
plt.tick_params(axis="both", which="minor", labelsize=16)
plt.show()
```





## Borrow a device from the system

```
In [5]: dev = qml.device("strawberryfields.fock", wires=1, cutoff_dim=10)
```

## Define function

```
In [7]: def layer(v):  
        # Matrix multiplication of input layer  
        qml.Rotation(v[0], wires=0)  
        qml.Squeezing(v[1], 0.0, wires=0)  
        qml.Rotation(v[2], wires=0)  
        # Bias  
        qml.Displacement(v[3], 0.0, wires=0)  
        # Element-wise nonlinear transformation  
        qml.Kerr(v[4], wires=0)
```

```
In [8]: @qml.qnode(dev)  
def quantum_neural_net(var, x=None):  
    # Encode input x into quantum state  
    qml.Displacement(x, 0.0, wires=0)  
    # "layer" subcircuits  
    for v in var:  
        layer(v)  
    return qml.expval(qml.X(0))
```

## qml.Rotation

```
class Rotation(phi, wires, do_queue=True, id=None)
```

[\[source\]](#)

Bases: `pennylane.operation.CVOperation`

Phase space rotation.

$$R(\phi) = \exp(i\phi\hat{a}^\dagger\hat{a}) = \exp\left(i\frac{\phi}{2}\left(\frac{\hat{x}^2 + \hat{p}^2}{\hbar} - \hat{1}\right)\right).$$

Details:

- Number of wires: 1
- Number of parameters: 1
- Gradient recipe:  $\frac{d}{dr}f(R(r)) = \frac{1}{2}[f(R(\phi + \pi/2)) - f(R(\phi - \pi/2))]$  where  $f$  is an expectation value depending on  $R(r)$ .
- Heisenberg representation:

$$M = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix}$$

## qml.Squeezing

```
class Squeezing(r, phi, wires, do_queue=True, id=None)
```

[\[source\]](#)

Bases: `pennylane.operation.CVOperation`

Phase space squeezing.

$$S(z) = \exp\left(\frac{1}{2}(z^* \hat{a}^2 - z \hat{a}^{\dagger 2})\right).$$

where  $z = r e^{i\phi}$ .

Details:

- Number of wires: 1
- Number of parameters: 2
- Gradient recipe:  $\frac{d}{dr} f(S(r, \phi)) = \frac{1}{2 \sinh s} [f(S(r + s, \phi)) - f(S(r - s, \phi))]$ , where  $s$  is an arbitrary real number (0.1 by default) and  $f$  is an expectation value depending on  $S(r, \phi)$ .
- Heisenberg representation:

$$M = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cosh r - \cos \phi \sinh r & -\sin \phi \sinh r \\ 0 & -\sin \phi \sinh r & \cosh r + \cos \phi \sinh r \end{bmatrix}$$

## qml.Displacement

```
class Displacement(a, phi, wires, do_queue=True, id=None)
```

[\[source\]](#)

Bases: `pennylane.operation.CVOperation`

Phase space displacement.

$$D(a, \phi) = D(\alpha) = \exp(\alpha \hat{a}^\dagger - \alpha^* \hat{a}) = \exp\left(-i \sqrt{\frac{2}{\hbar}} (\operatorname{Re}(\alpha) \hat{p} - \operatorname{Im}(\alpha) \hat{x})\right).$$

where  $\alpha = ae^{i\phi}$  has magnitude  $a \geq 0$  and phase  $\phi$ . The result of applying a displacement to the vacuum is a coherent state  $D(\alpha)|0\rangle = |\alpha\rangle$ .

### Details:

- Number of wires: 1
- Number of parameters: 2
- Gradient recipe:  $\frac{d}{da} f(D(a, \phi)) = \frac{1}{2s} [f(D(a + s, \phi)) - f(D(a - s, \phi))]$ , where  $s$  is an arbitrary real number (0.1 by default) and  $f$  is an expectation value depending on  $D(a, \phi)$ .
- Heisenberg representation:

$$M = \begin{bmatrix} 1 & 0 & 0 \\ 2a \cos \phi & 1 & 0 \\ 2a \sin \phi & 0 & 1 \end{bmatrix}$$

## qml.Kerr

```
class Kerr(kappa, wires, do_queue=True, id=None)
```

[\[source\]](#)

Bases: `pennylane.operation.CVOperation`

Kerr interaction.

$$K(\kappa) = e^{i\kappa\hat{n}^2}.$$

### Details:

- Number of wires: 1
- Number of parameters: 1
- Gradient recipe: None (uses finite difference)

## Define function2

```
In [9]: def square_loss(labels, predictions):  
        loss = 0  
        for l, p in zip(labels, predictions):  
            loss = loss + (l - p) ** 2  
        loss = loss / len(labels)  
        return loss
```

```
In [10]: def cost(var, features, labels):  
         preds = [quantum_neural_net(var, x=x) for x in features]  
         return square_loss(labels, preds)
```

- var = weight matrix, features = X value, labels = Y value

# Initial value of weight

```
In [12]: np.random.seed(0)
num_layers = 6
var_init = 0.05 * np.random.randn(num_layers, 5)
print(var_init)

[[ 0.08820262  0.02000786  0.0489369  0.11204466  0.0933779 ]
 [-0.04886389  0.04750442 -0.00756786 -0.00516094  0.02052993]
 [ 0.00720218  0.07271368  0.03805189  0.00608375  0.02219316]
 [ 0.01668372  0.07470395 -0.01025791  0.01565339 -0.04270479]
 [-0.12764949  0.03268093  0.04322181 -0.03710825  0.11348773]
 [-0.07271828  0.00228793 -0.00935919  0.07663896  0.07346794]]

In [46]: var_init_one = var_init[0,:];
print(var_init_one)
var_init_two = var_init[0,2,:];
print(var_init_two)
var_init_four = var_init[0,4,:];
print(var_init_four)
var_init_six = var_init
print(var_init)

[[0.08820262 0.02000786 0.0489369 0.11204466 0.0933779 ]
 [ 0.08820262 0.02000786 0.0489369 0.11204466 0.0933779 ]
 [-0.04886389 0.04750442 -0.00756786 -0.00516094 0.02052993]
 [ 0.08820262 0.02000786 0.0489369 0.11204466 0.0933779 ]
 [-0.04886389 0.04750442 -0.00756786 -0.00516094 0.02052993]
 [ 0.00720218 0.07271368 0.03805189 0.00608375 0.02219316]
 [ 0.01668372 0.07470395 -0.01025791 0.01565339 -0.04270479]
 [ 0.08820262 0.02000786 0.0489369 0.11204466 0.0933779 ]
 [-0.04886389 0.04750442 -0.00756786 -0.00516094 0.02052993]
 [ 0.00720218 0.07271368 0.03805189 0.00608375 0.02219316]
 [ 0.01668372 0.07470395 -0.01025791 0.01565339 -0.04270479]
 [-0.12764949 0.03268093 0.04322181 -0.03710825 0.11348773]
 [-0.07271828 0.00228793 -0.00935919 0.07663896 0.07346794]]
```



# Table of Contents

---

## Cosine function fitting

- Review of the neural network

- Construction of data and function

- Cosine function fitting by QNN

## Distinguish handwritten 1 and 7 by QNN

- Setting of the problem

- Training by QNN

- Comparison with CNN

```
In [37]: import time
start = time.time() # 시작 시간 저장

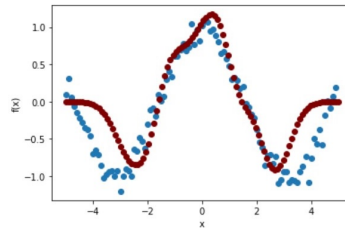
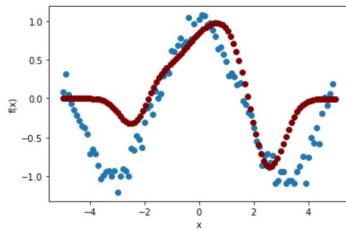
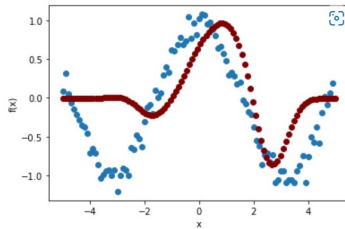
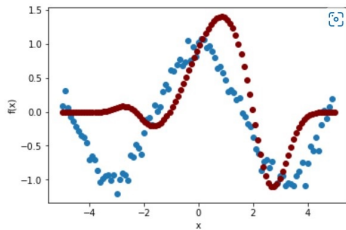
opt = AdamOptimizer(0.01, beta1=0.9, beta2=0.999)
var = var_init_one
for it in range(50):
    var = opt.step(lambda v: cost(v, X, Y), var)
    print("Iter: {:5d} | Cost: {:.7f} ".format(it + 1, cost(var, X, Y)))

print("time :", time.time() - start) # 현재시각 - 시작시간 = 실행 시간
```

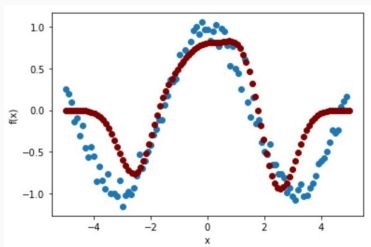
|       |   |                 |
|-------|---|-----------------|
| Iter: | 1 | Cost: 1.5446380 |
| Iter: | 2 | Cost: 1.2842563 |
| Iter: | 3 | Cost: 1.1486733 |
| Iter: | 4 | Cost: 1.1085822 |
| Iter: | 5 | Cost: 1.1162507 |
| Iter: | 6 | Cost: 1.1321445 |

$$w_{ij}^{n+1} = w_{ij}^n - \eta \nabla E = w_{ij}^n - \eta \frac{\partial C(w_1, w_2)}{\partial w_{ij}}$$

# Result1



## Result2



|            |           |           |           |           |           |
|------------|-----------|-----------|-----------|-----------|-----------|
| # of layer | 1         | 2         | 4         | 6         | 4(500)    |
| Time       | 81.77     | 169.92    | 411.87    | 395.17    | 2508.20   |
| Last Cost  | 0.3298956 | 0.2769603 | 0.2052505 | 0.1288593 | 0.1296430 |

- The model learned smoothness from the noise data.
- Performance is stable around 6-layer.

# Table of Contents

## Cosine function fitting

- Review of the neural network

- Construction of data and function

- Cosine function fitting by QNN

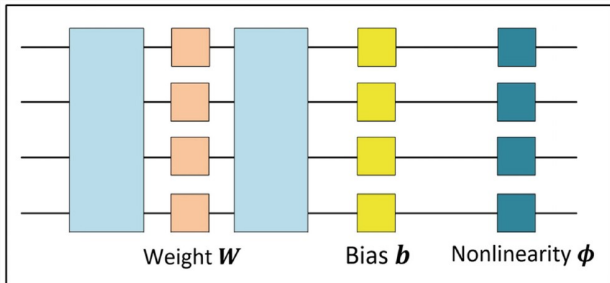
## Distinguish handwritten 1 and 7 by QNN

- Setting of the problem

- Training by QNN

- Comparison with CNN

# Setting of the problem



Input: Hand written Image data 1 or 7

Output: 1-True or 7-False

## Warning

- Version is important.
- Building an environment is the most important thing.

```
[ ] !pip install tensorflow==2.7.0
```

Looking in indexes: <https://pypi.org/simple>, <https://us-python.pkg.dev/colab-wheels/public/simple/>

Collecting tensorflow==2.7.0

Downloading [https://us-python.pkg.dev/colab-wheels/public/tensorflow/tensorflow-2.7.0%2Bzzcolab20220506150900-cp37-cp37m-linux\\_x86\\_64.whl](https://us-python.pkg.dev/colab-wheels/public/tensorflow/tensorflow-2.7.0%2Bzzcolab20220506150900-cp37-cp37m-linux_x86_64.whl)  
/ 665.5 MB 278 kB/s

Requirement already satisfied: numpy>=1.14.5 in /usr/local/lib/python3.7/dist-packages (from tensorflow==2.7.0) (1.21.6)

Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.7/dist-packages (from tensorflow==2.7.0) (3.3.0)

Requirement already satisfied: absl-py>=0.4.0 in /usr/local/lib/python3.7/dist-packages (from tensorflow==2.7.0) (1.0.0)

Requirement already satisfied: grpcio&lt;2.0.&gt;=1.24.3 in /usr/local/lib/python3.7/dist-packages (from tensorflow==2.7.0) (1.46.1)

Collecting gast&lt;0.5,0,&gt;=0.2,1

Downloading gast-0.4.0-py3-none-any.whl (9.8 kB)

Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.7/dist-packages (from tensorflow==2.7.0) (1.15.0)

Requirement already satisfied: tensorflow-io-gcs-filesystem&gt;=0.21.0 in /usr/local/lib/python3.7/dist-packages (from tensorflow==2.7.0) (0.26

Requirement already satisfied: wrapt&gt;=1.11.0 in /usr/local/lib/python3.7/dist-packages (from tensorflow==2.7.0) (1.14.1)

Requirement already satisfied: protobuf&gt;=3.9.2 in /usr/local/lib/python3.7/dist-packages (from tensorflow==2.7.0) (3.17.3)

Collecting tensorflow-estimator&lt;2.8,&gt;=2.7.0rc0

Downloading tensorflow\_estimator-2.7.0-py2.py3-none-any.whl (463 kB)

463 kB 7.4 MB/s

Requirement already satisfied: h5py>=2.9.0 in /usr/local/lib/python3.7/dist-packages (from tensorflow==2.7.0) (3.1.0)

Requirement already satisfied: astunparse&gt;=1.6.0 in /usr/local/lib/python3.7/dist-packages (from tensorflow==2.7.0) (1.6.3)

Requirement already satisfied: tensorboard==2.6 in /usr/local/lib/python3.7/dist-packages (from tensorflow==2.7.0) (2.6.0)



## Warning

```
[ ] !pip install tensorflow-quantum
```

Looking in indexes: <https://pypi.org/simple>, <https://us-python.pkg.dev/colab-wheels/public/simple/>

Collecting tensorflow-quantum

Downloading tensorflow\_quantum-0.6.1-cp37-cp37m-manylinux\_2\_12\_x86\_64\_manylinux2010\_x86\_64.whl (10.5 MB)

10.5 MB 6.9 MB/s

Collecting google-auth==1.18.0

Downloading google\_auth-1.18.0-py2.py3-none-any.whl (90 kB)

90 kB 8.3 MB/s

Collecting cirq-google&gt;=0.13.1

Downloading circ\_google-0.14.1-py3-none-any.whl (541 kB)

 541 kB 55.3 MB/s

Collecting google-api-core==1.21.0

Downloading google\_api\_core-1.21.0-py2.py3-none-any.whl (90 kB)

90 kB 10.3 MB/s

Collecting sympy==1.8

Downloading sympy-1.8-py3-none-any.whl (6.1 MB)

6.1 MB 39.3 MB/s

Requirement already satisfied: protobuf==3.17.3 in /usr/local/lib/python3.7/dist-packages (from tensorflow-quantum)

Collecting googleapis-common-protos==1.52.0

Downloading googleapis\_common\_protos-1.52.0-py2.py3-none-any.whl (100 kB)

100 kB 10.5 MB/s

Collecting cirq-core&gt;=0.13.1

Downloading cirq\_core-0.14.1-py3-none-any.whl (1.8 MB)

1.8 MB 48.6 MB/s

# Import

```
[1] import tensorflow as tf
import tensorflow_quantum as tfq
import cirq
import sympy
import numpy as np
import seaborn as sns
import collections
```

```
[2] %matplotlib inline
import matplotlib.pyplot as plt
from cirq.contrib.svg import SVGCircuit
```

# Data manipulation

```
[3] (x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
x_train, x_test = x_train[... , np.newaxis]/255.0, x_test[... ,
np.newaxis]/255.0
print("Number of original training examples:", len(x_train))
print("Number of original test examples:", len(x_test))
```

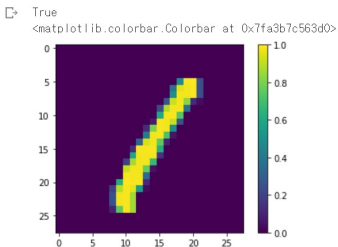
Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>  
11493376/11490434 [=====] - 0s 0us/step  
11501568/11490434 [=====] - 0s 0us/step  
Number of original training examples: 60000  
Number of original test examples: 10000

```
[5] def filter_17(x, y):
    keep = (y == 1) | (y == 7)
    x, y = x[keep], y[keep]
    y = y == 1
    return x,y
x_train, y_train = filter_17(x_train, y_train)
x_test, y_test = filter_17(x_test, y_test)
print("Number of filtered training examples:", len(x_train))
print("Number of filtered test examples:", len(x_test))
```

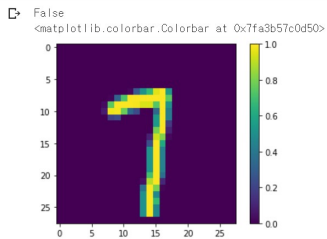
Number of filtered training examples: 13007  
Number of filtered test examples: 2163

# Data manipulation

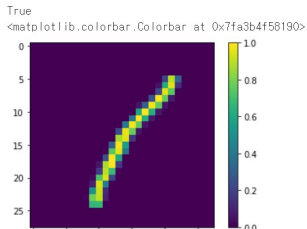
```
print(y_train[0])  
plt.imshow(x_train[0, :, :, 0])  
plt.colorbar()
```



```
print(y_train[5001])  
plt.imshow(x_train[5001, :, :, 0])  
plt.colorbar()
```



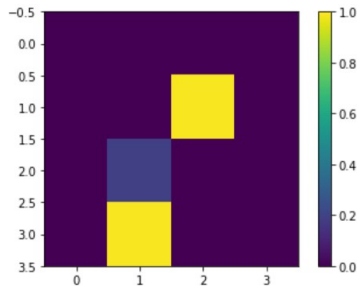
```
[29] print(y_train[13006])  
plt.imshow(x_train[13006, :, :, 0])  
plt.colorbar()
```



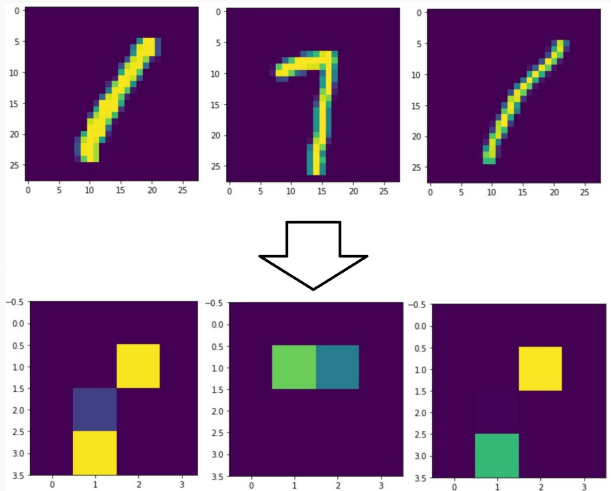
# Data manipulation

```
▶ x_train_small = tf.image.resize(x_train, (4,4)).numpy()  
x_test_small = tf.image.resize(x_test, (4,4)).numpy()  
print(y_train[0])  
plt.imshow(x_train_small[0,:,:,:], vmin=0, vmax=1)  
plt.colorbar()
```

```
↳ True  
<matplotlib.colorbar.Colorbar at 0x7fa3b771fb90>
```

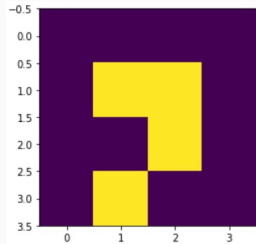
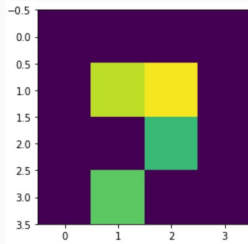
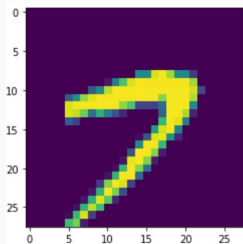


# Data manipulation

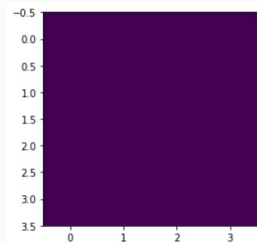
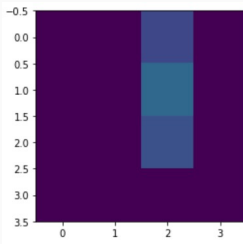
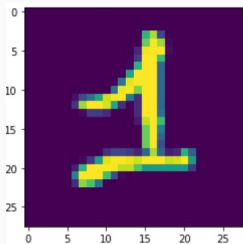


# Data manipulation

```
[74] THRESHOLD = 0.5  
x_train_bin = np.array(x_train_small > THRESHOLD, dtype=np.float32)  
x_test_bin = np.array(x_test_small > THRESHOLD, dtype=np.float32)
```



# Data manipulation





# Table of Contents

## Cosine function fitting

- Review of the neural network

- Construction of data and function

- Cosine function fitting by QNN

## Distinguish handwritten 1 and 7 by QNN

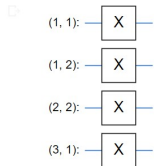
- Setting of the problem

- Training by QNN**

- Comparison with CNN

# Convert to circuit

```
def convert_to_circuit(image):  
    values = np.ndarray.flatten(image)  
    qubits = cirq.GridQubit.rect(4, 4)  
    circuit = cirq.Circuit()  
    for i, value in enumerate(values):  
        if value:  
            circuit.append(cirq.X(qubits[i]))  
    return circuit  
x_train_circ = [convert_to_circuit(x) for x in x_train_bin]  
x_test_circ = [convert_to_circuit(x) for x in x_test_bin]  
SVGCircuit(x_train_circ[21])
```



```
[79] bin_img = x_train_bin[21,:,:;0]  
indices = np.array(np.where(bin_img)).T  
indices
```

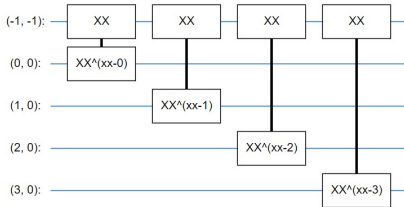
```
array([[1, 1],  
       [1, 2],  
       [2, 2],  
       [3, 1]])
```

# Building a layer

```
[14] x_train_tfqcirc = tfq.convert_to_tensor(x_train_circ)
     x_test_tfqcirc = tfq.convert_to_tensor(x_test_circ)

[15] class CircuitLayerBuilder():
     def __init__(self, data_qubits, readout):
         self.data_qubits = data_qubits
         self.readout = readout
     def add_layer(self, circuit, gate, prefix):
         for i, qubit in enumerate(self.data_qubits):
             symbol = sympy.Symbol(prefix + '-' + str(i))
             circuit.append(gate(qubit, self.readout)**symbol)
demo_builder = CircuitLayerBuilder(data_qubits = cirq.GridQubit.rect(4,1),readout=cirq.GridQubit(-1,-1))

circuit = cirq.Circuit()
demo_builder.add_layer(circuit, gate = cirq.XX, prefix='xx')
SVGCircuit(circuit)
```



## Creat quantum circuit

```
[16] def create_quantum_model():
    data_qubits = cirq.GridQubit.rect(4, 4) # a 4x4 grid
    readout = cirq.GridQubit(-1, -1) # a single qubit at [-1,-1]
    circuit = cirq.Circuit()
    # Prepare the readout qubit.
    circuit.append(cirq.X(readout))
    circuit.append(cirq.H(readout))
    builder = CircuitLayerBuilder(
        data_qubits = data_qubits, readout=readout)
    # Then add layers (experiment by adding more).
    builder.add_layer(circuit, cirq.XX, "xx1")
    builder.add_layer(circuit, cirq.ZZ, "zz1")
    # Finally, prepare the readout qubit.
    circuit.append(cirq.H(readout))
    return circuit, cirq.Z(readout)
model_circuit, model_readout = create_quantum_model()
```

# Cost function

```
[19] # Build the Keras model.  
model = tf.keras.Sequential([tf.keras.layers.Input(shape=(), dtype=tf.string), tf.keras.layers.PQC(model_circuit, model_readout),])
```



```
y_train_hinge = 2.0*y_train-1.0  
y_test_hinge = 2.0*y_test-1.0
```

```
[22] def hinge_accuracy(y_true, y_pred):  
    y_true = tf.squeeze(y_true) > 0.0  
    y_pred = tf.squeeze(y_pred) > 0.0  
    result = tf.cast(y_true == y_pred, tf.float32)  
    return tf.reduce_mean(result)
```

```
model.compile(  
    loss=tf.keras.losses.Hinge(),  
    optimizer=tf.keras.optimizers.Adam(),  
    metrics=[hinge_accuracy])  
print(model.summary())
```

Model: "sequential"

| Layer (type) | Output Shape | Param # |
|--------------|--------------|---------|
| pqc (PQC)    | (None, 1)    | 32      |

=====  
Total params: 32  
Trainable params: 32  
Non-trainable params: 0

=====  
None

# QNN 4 × 4 pixel result

```
▶ EPOCHS = 3
  BATCH_SIZE = 32
  NUM_EXAMPLES = len(x_train_tfcirc)
  x_train_tfcirc_sub = x_train_tfcirc[:NUM_EXAMPLES]
  y_train_hinge_sub = y_train_hinge[:NUM_EXAMPLES]
  qnn_history = model.fit(
    x_train_tfcirc_sub, y_train_hinge_sub,
    batch_size=32,
    epochs=EPOCHS,
    verbose=1,
    validation_data=(x_test_tfcirc, y_test_hinge))
  qnn_results = model.evaluate(x_test_tfcirc, y_test)

🔗 Epoch 1/3
407/407 [=====] - 1485s 4s/step - loss: 0.6189 - hinge_accuracy: 0.7779 - val_loss: 0.3883 - val_hinge_accuracy: 0.8624
Epoch 2/3
407/407 [=====] - 1252s 3s/step - loss: 0.3980 - hinge_accuracy: 0.8591 - val_loss: 0.3805 - val_hinge_accuracy: 0.8638
Epoch 3/3
407/407 [=====] - 1269s 3s/step - loss: 0.3941 - hinge_accuracy: 0.8520 - val_loss: 0.3787 - val_hinge_accuracy: 0.8564
68/68 [=====] - 50s 723ms/step - loss: 0.3787 - hinge_accuracy: 0.8564
```

# Table of Contents

## Cosine function fitting

- Review of the neural network

- Construction of data and function

- Cosine function fitting by QNN

## Distinguish handwritten 1 and 7 by QNN

- Setting of the problem

- Training by QNN

- Comparison with CNN

```
def create_classical_model():  
    # A simple model based off LeNet from https://keras.io/examples/mnist\_cnn/  
    model = tf.keras.Sequential()  
    model.add(tf.keras.layers.Conv2D(32, [3, 3], activation='relu',  
input_shape=(28,28,1)))  
    model.add(tf.keras.layers.Conv2D(64, [3, 3], activation='relu'))  
    model.add(tf.keras.layers.MaxPooling2D(pool_size=(2, 2)))  
    model.add(tf.keras.layers.Dropout(0.25))  
    model.add(tf.keras.layers.Flatten())  
    model.add(tf.keras.layers.Dense(128, activation='relu'))  
    model.add(tf.keras.layers.Dropout(0.5))  
    model.add(tf.keras.layers.Dense(1))  
    return model  
model = create_classical_model()  
model.compile(loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),  
              optimizer=tf.keras.optimizers.Adam(),  
              metrics=['accuracy'])  
model.summary()
```



## CNN $28 \times 28$ pixel result

```
✓ [56] model.fit(x_train,  
1분  y_train,  
      batch_size=128,  
      epochs=1,  
      verbose=1,  
      validation_data=(x_test, y_test))  
cnn_results = model.evaluate(x_test, y_test)  
  
102/102 [=====] - 65s 612ms/step - loss: 0.0444 - accuracy: 0.9799 - val_loss: 0.0200 - v  
68/68 [=====] - 3s 40ms/step - loss: 0.0200 - accuracy: 0.9935
```

```
[57] def create_fair_classical_model():  
    # A simple model based off LeNet from https://keras.io/examples/mnist\_cnn/  
    model = tf.keras.Sequential()  
    model.add(tf.keras.layers.Flatten(input_shape=(4,4,1)))  
    model.add(tf.keras.layers.Dense(2, activation='relu'))  
    model.add(tf.keras.layers.Dense(1))  
    return model  
model = create_fair_classical_model()  
model.compile(loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),  
              optimizer=tf.keras.optimizers.Adam(),  
              metrics=['accuracy'])  
model.summary()
```

## CNN $4 \times 4$ pixel result

✓  
10  
초



```
model.fit(x_train_bin,  
          y_train,  
          batch_size=128,  
          epochs=20,  
          verbose=2,  
          validation_data=(x_test_bin, y_test))  
fair_nn_results = model.evaluate(x_test_bin, y_test)
```

Epoch 17/20

102/102 - 0s - loss: 0.3062 - accuracy: 0.8691 - val\_loss: 0.2893 - val\_accuracy: 0.8816 - 347ms/epoch - 3ms/step

Epoch 18/20

102/102 - 0s - loss: 0.3055 - accuracy: 0.8712 - val\_loss: 0.2885 - val\_accuracy: 0.8807 - 369ms/epoch - 4ms/step

Epoch 19/20

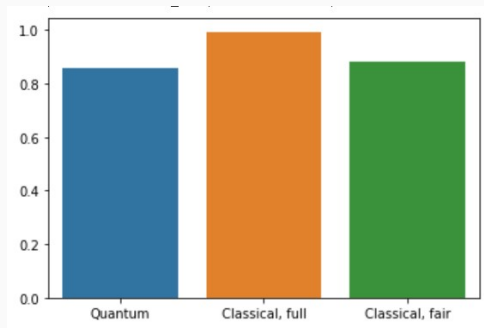
102/102 - 0s - loss: 0.3050 - accuracy: 0.8655 - val\_loss: 0.2881 - val\_accuracy: 0.8816 - 419ms/epoch - 4ms/step

Epoch 20/20

102/102 - 0s - loss: 0.3047 - accuracy: 0.8715 - val\_loss: 0.2874 - val\_accuracy: 0.8807 - 351ms/epoch - 3ms/step

68/68 [=====] - 0s 3ms/step - loss: 0.2874 - accuracy: 0.8807

## Comparison of accuracy



- For classical dataset, CNN  $\gg$  QNN.
- Let us get insights from QNN on the classical data. For quantum data (or high dimensional data), having superposition and entanglement, QNN will exceed the performance of CNN.
- Next-generation quantum computing systems will bring unprecedented computational power.

**THANK YOU FOR LISTENING**